Контейнеры, итераторы и их применение

Александра Волосова,
к.т.н., доцент кафедры
ИУ5

Презентация разработана в рамках гранта «Грант на обучение студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий. Договор № 70-2025-000850 с АНО «Аналитический центр при Правительстве Российской Федерации»

Часть 1

STL

STL (Standard Template Library) – стандартная библиотека шаблонов

Парадигмы:

1. Объектно-ориентированное программирование

Контейнеры — это **классы**. Они инкапсулируют данные и предоставляют методы (push_back, size, begin, end).

Иерархия потоков ввода-вывода (iostream, fstream, stringstream) построена на **классическом наследовании** и **полиморфизме**.

Умные указатели —**классы** с конструкторами, деструкторами, перегрузкой операторов

2. Обобщенное программирование (ОП) — главная идея STL: Один код — для любых типов данных.

Парадигма программирования - описание данных и алгоритмов, которое можно применять к **различным типам данных**, не меняя само описание

Главные принципы:

Алгоритмы не зависят от типов контейнеров. Один алгоритм **std::sort** работает с vector, deque, обычным массивом — через итераторы.

Итераторы — обобщение указателей, позволяющее единообразно обращаться к разным структурам данных. Это достигается через **шаблоны**, а не через наследование.

ОП реализуется в C++ с помощью шаблонов классов и функций

3. Функциональное программирование (начиная с C++11 и далее)

Лямбды, std::function, алгоритмы, принимающие функциональные объекты.

Период	Язык	Вклад
1970-е	Clu, Ада	Дженерики (обобщённые функции)
1970-80-е	ML	Параметрический полиморфизм
1990-е	C++	Шаблоны (templates), STL
2000-е	Java, C#, D, Delphi	Дженерики
2010-е	Python, Nim	Поддержка обобщений

Методология обобщенного программирования

Обобщённое программирование - разделение **структур данных** и **алгоритмов** через использование **абстрактных описаний требований**.

Вместо описания отдельного типа применяется **описание семейства типов**, имеющих:

- общий интерфейс
- семантическое поведение

Концепция - **ключевое понятие ОП**: набор требований, описывающий интерфейс и семантическое поведение

Алгоритм, написанный в **обобщённом стиле** может применяться для любых типов, подходящих к нему по своим концепциями. Такая возможность называется **полиморфизмом**

Компоненты концепции:

- **Допустимые выражения** — выражения языка программирования, которые должны успешно компилироваться для типов, удовлетворяющих требованиям концепции
- **Ассоциированные типы** — вспомогательные типы, имеющие некоторое отношение к моделирующему концепцию типу
- **Инварианты** — такие характеристики типов, которые должны быть постоянно верны во время исполнения. Обычно выражаются в виде предусловий и постусловий. Невыполнение предусловия влечёт непредсказуемость соответствующей операции и может привести к ошибкам
- **Гарантии сложности** — максимальное время выполнения допустимого выражения или максимальные требования к различным ресурсам в ходе выполнения этого выражения

Обобщенное программирование в C++. Шаблон функции

```
// Шаблон функции
template <typename T>
T max(T x, T y) {
   if (x < y)
      return y;
   else
      return x;
}

// Использование
int a = max(10, 15);              // T = int
double b = max(3.14, 2.71);  // T = double
string c = max("abc", "xyz"); // T = string
```

```
#include <iostream>
#include <string>
template <typename T>
T max(T x, T y) {
   if (x < y)
      return y;
   else
      return x;
}
int main() {
   int a = max(10, 15);                    // T = int
   double b = max(3.14, 2.71);         // T = double
   std::string c = max("abc", "xyz");   // T = std::string
   std::cout << "max(10, 15) = " << a << std::endl;
   std::cout << "max(3.14, 2.71) = " << b << std::endl;
   std::cout << "max(\"abc\", \"xyz\") = " << c <<std::endl;
   return 0;
}
```

```
max(10, 15) = 15
max(3.14, 2.71) = 3.14
max("abc", "xyz") = abc
```

Обобщенное программирование в C++. Шаблон класса

```
// Шаблон класса "Связный список"
template <class T>
class List {
public:
   void Add(const T& Element);
   bool Find(const T& Element);
   // ...
};

// Использование
List<int>    intList;      // список int
List<string> stringList;   // список string
List<Person> peopleList;   // список Person
```

```
20 10
hello world
```

```
#include <iostream>
#include <string>
template <class T>
class List {
   struct Node { T data; Node* next; };
   Node* head = nullptr;
public:
   void Add(const T& val) { head = new Node{val, head}; }
   bool Find(const T& val) {
      for (auto p = head; p; p = p->next)
         if (p->data == val) return true;
      return false; }
   void Print() {
      for (auto p = head; p; p = p->next)
         std::cout << p->data << " ";
      std::cout << std::endl; }
   ~List() { while (head) { Node* t = head; head = head->next; delete t; } } };
int main() {
   List<int> intList;          // список int
   intList.Add(10);
   intList.Add(20);
   intList.Print();            // 20 10
   List<std::string> strList;  // список string
   strList.Add("world");
   strList.Add("hello");
   strList.Print();            // hello world }
```

STL: общая архитектура

Три основных компонента (Александр Степанов):

1. Контейнеры — хранение данных
2. Итераторы — доступ к данным
3. Алгоритмы — обработка данных

Типы контейнеров:

- Последовательные (sequence): array, vector, deque, list, forward_list
- Ассоциативные (associative): map, set, multimap, multiset
- Неупорядоченные (unordered): unordered_map, unordered_set
- Адаптеры: stack, queue, priority_queue

Контейнеры. std::string: строки как контейнеры

std::string — специализация std::basic_string<char>
Наследует все методы vector и добавляет строковые операции

```
std::string str = "Hello";

// Методы как у vector:
• str.push_back('!')      // "Hello!"
• str.pop_back()          // "Hello"
• str.size() / str.length()
• str.capacity()
• str.reserve(100)
```

```
// Специфичные строковые методы:
• str.substr(1, 3)       // "ell"
• str.find("ll")         // 2
• str.replace(1, 3, "i")  // "Hio"
• str.compare("Hello")
• str += " World"        // "Hello World"
• std::getline(std::cin, str)
// string как контейнер символов
for (char c : str) { /* ... */ }
std::sort(str.begin(), str.end());
```

Современные особенности (C++17/20):
- std::string_view для невладеющих строк
- starts_with(), ends_with() (C++20)
- contains() (C++23)

Оптимизация малых строк (Small String Optimization — SSO):
- Короткие строки хранятся в самом объекте, без выделения памяти
- Обычно до 15-23 символов (зависит от реализации)

Рекомендация экспертов: **Используйте std::string вместо C-строк везде, где это возможно**

Контейнеры. array и vector: основные методы

std::array (C++11) — статический массив фиксированного размера

```
std::array<int, 5> arr = {1, 2, 3, 4, 5};
// Методы:
• arr.size()      // 5 (константа времени компиляции)
• arr.at(2)       // 3 с проверкой границ
• arr[2]          // 3 без проверки
• arr.front()     // 1
• arr.back()      // 5
• arr.data()      // указатель на сырые данные
• arr.fill(0)     // заполнить нулями
```

std::vector — динамический массив

```
std::vector<int> vec = {1, 2, 3};
// Основные методы:
• vec.push_back(4)     // добавление в конец
• vec.pop_back()       // удаление с конца
• vec.insert(pos, val) // вставка
• vec.erase(pos)       // удаление
• vec.reserve(n)       // резервирование памяти
• vec.shrink_to_fit()  // освобождение лишней памяти
• vec.emplace_back(args) // создание на месте
```

Особенности vector:

- Элементы хранятся непрерывно в памяти
- Быстрый случайный доступ O(1)
- Медленные вставка/удаление в середине O(n)
- При увеличении ёмкости происходит перераспределение памяти

Рекомендация экспертов: Предпочитайте vector всем другим контейнерам, если нет веской причины использовать другой

Контейнеры. push_back и emplace_back

push_back — добавление копии/перемещение
emplace_back — создание на месте

```
#include <iostream>
#include <vector>
#include <string>
class Widget {
   std::string name;
   int id;
public:
   Widget(std::string n, int i) : name(std::move(n)), id(i) {
      std::cout << "Widget created: " << name << ", " << id << std::endl;
   }
};
int main() {
   std::vector<Widget> widgets;
   std::cout << "--- push_back with temporary ---" << std::endl;
// push_back создает временный объект, затем копирует/перемещает
   widgets.push_back(Widget("test", 1)); // Временный объект  и перемещение
   std::cout << "--- push_back with initializer list ---" << std::endl;
// emplace_back создает объект непосредственно в памяти вектора
   widgets.push_back({"test", 1}); // Прямой вызов конструктора
   std::cout << "--- emplace_back ---" << std::endl;
   widgets.emplace_back("test", 1);
}
```

Your Output

```
--- push_back with temporary ---
Widget created: test, 1
--- push_back with initializer list ---
Widget created: test, 1
--- emplace_back ---
Widget created: test, 1
```

Контейнеры. push_back и emplace_back

Реаллокация вектора:

```
#include <iostream>
#include <vector>
int main() {
   std::vector<int> vec;
   std::cout << "Начальная capacity: " << vec.capacity() << std::endl;
   vec.reserve(3);         // Резервируем память для 3 элементов
   std::cout << "После reserve(3): capacity = " << vec.capacity() << std::endl;
   vec.push_back(1); // Без реаллокации
   vec.push_back(2);
   vec.push_back(3);
   std::cout << "После 3 push_back: capacity = " << vec.capacity() << std::endl;
   vec.push_back(4);       // Реаллокация,  capacity увеличивается (обычно в 1.5-2
раза)
   std::cout << "После 4-го push_back: capacity = " << vec.capacity() << std::endl;
   std::cout << "Новая capacity: " << vec.capacity()  << " (обычно в 1.5-2 раза
больше)" << std::endl; }
```

Your Output

```
Начальная capacity: 0
После reserve(3): capacity = 3
После 3 push_back: capacity = 3
После 4-го push_back: capacity = 6
Новая capacity: 6 (обычно в 1.5-2 раза больше)
```

Контейнеры. push_back и emplace_back

Преимущества emplace_back:

- Позволяет избежать создания временного объекта
- Может быть более эффективным для немобильных типов*
- Позволяет использовать explicit конструкторы

```
* struct NonMovable {
      NonMovable(int x) : value(x) {}
      NonMovable(const NonMovable&) = default;      // копирование разрешено
      NonMovable(NonMovable&&) = delete;            // перемещение класса  запрещено
      int value;
  };
  //попытка добавить объект  в вектор:
  std::vector<NonMovable> vec;
  // ошибка компиляции при попытке переместить созданный  временный объект в вектор
  vec.push_back(NonMovable(42));
  // перемещение не требуется, так как объект создается сразу  нужном месте
  vec.emplace_back(42);
```

push_back следует использовать уже с готовым объектом и в том случае, когда нужна ясность кода

Контейнеры. Placement new

Placement new — создание объекта в заранее выделенной памяти

```
#include <new>
#include <iostream>
class Widget
public:
   Widget() { std::cout << "Constructor\n"; }
   ~Widget() { std::cout << «Destructor\n"; }
};
int main() {
   char buffer[sizeof(Widget)];      // Неинициализированная память
   Widget* w = new (buffer) Widget(); // placement new
   w->~Widget();                      // явный вызов деструктора
   // память не освобождается
}
```

Your Output

```
Constructor
Destructor
```

Использование в vector::emplace_back:

1. Проверка capacity
2. Если нужно — reallocation
3. Placement new в следующей позиции
4. Увеличение size

Контейнеры. Аллокаторы – управление памятью для контейнера

```
#include <iostream>
#include <vector>
// Пользовательский аллокатор
template<typename T>
class CustomAllocator {
public:
   using value_type = T;
   T* allocate(size_t n) {
      std::cout << "Allocate " << n << " elements" << std::endl;
      return static_cast<T*>(::operator new(n * sizeof(T)));
   }
   void deallocate(T* p, size_t n) noexcept {
      std::cout << "Deallocate " << n << " elements" << std::endl;
      ::operator delete(p);
   }
  template<typename U, typename... Args>
   void construct(U* p, Args&&... args) {
      std::cout << "Construct element" << std::endl;
      new (p) U(std::forward<Args>(args)...);  // Placement new!
   }
  template<typename U>
   void destroy(U* p) noexcept {
      std::cout << "Destroy element" << std::endl;
      p->~U();  // Явный вызов деструктора
   }
 // Другие методы: rebind, address, max_size... } ;
```

```
int main() {
   std::vector<int, CustomAllocator<int>> vec;
   vec.push_back(10);
   vec.push_back(20);
   vec.push_back(30);
   return 0;
}
```

Your Output

```
Allocate 1 elements
Construct element
Allocate 2 elements
Construct element
Construct element
Destroy element
Deallocate 1 elements
Allocate 4 elements
Construct element
```

```
Construct element
Destroy element
Destroy element
Deallocate 2 elements
Destroy element
Destroy element
Destroy element
Deallocate 4 elements
```

Рекомендация экспертов: Аллокаторы позволяют отделить стратегию выделения памяти от логики контейнера

Современные аллокаторы (C++17):

- std::pmr::polymorphic_allocator
- Memory resources

Контейнеры. Аллокаторы

Аллокатор — стратегия выделения памяти для контейнеров

Ключевые требования к аллокаторам:

Должны быть шаблонными	template<typename T> class MyAllocator { ... };

Должны иметь rebind	template<typename U>
	struct rebind {
	  using other = MyAllocator<U>;
	};
	В C++17 rebind заменяется allocator_traits

Должны поддерживать копирование	Аллокаторы должны поддерживать копирование, чтобы контейнеры могли их передавать и хранить

Должны иметь typedef (value_type, pointer и т.д.)	using value_type = T;
	using pointer = T*;
	using const_pointer = const T*;
	using reference = T&;
	using const_reference = const T&;
	using size_type = std::size_t;
	using difference_type = std::ptrdiff_t;

Основные функции (C++11):

1. **allocate(n)** — выделить память для n объектов
2. **deallocate(p, n)** — освободить память
3. **construct(p, args...)** — создать объект
4. **destroy(p)** — уничтожить объект

Контейнеры. Аллокаторы

Пример аллокатора с пулом памяти. **Аллокатор управляет памятью вместо стандартного** new/delete**:**

```
#include <vector>
#include <iostream>
template<typename T>
class PoolAllocator {
   static std::vector<void*> pool; //инициализация пула
public:
   using value_type = T;
   T* allocate(size_t n) {
      if (n != 1) return static_cast<T*>(::operator new(n * sizeof(T)));
      if (!pool.empty()) {
         void* p = pool.back();
         pool.pop_back();
         return static_cast<T*>(p);
      }
      return static_cast<T*>(::operator new(sizeof(T)));
   }
   void deallocate(T* p, size_t n) noexcept {
      if (n == 1) pool.push_back(p); // Возвращаем память в пул
      else ::operator delete(p);
   }
};
template<typename T>
std::vector<void*> PoolAllocator<T>::pool;
```

```
int main() {
   std::vector<int, PoolAllocator<int>> v;
// Аллокатор выделяет память из пула
   for (int i = 0; i < 5; ++i) v.push_back(i);
// память сохраняется в пуле
   v.clear(); вызывается deallocate
//память выделяется из пула
   for (int i = 0; i < 5; ++i) v.push_back(i + 100);
   for (int x : v) std::cout << x << " ";
}
```

Your Output

```
100 101 102 103 104
```

Адаптеры над контейнерами

Адаптеры — обертки над другими контейнерами, предоставляющие специфичный интерфейс

```
1. std::stack — LIFO (Last In, First Out)
  template<typename T, typename Container = std::deque<T>>
  class stack {
     Container c;
  public:
     void push(const T& value) { c.push_back(value); }
     void pop() { c.pop_back(); }
     T& top() { return c.back(); }
     bool empty() const { return c.empty(); }
     size_t size() const { return c.size(); }
  };
  std::stack<int> s;                          // по умолчанию deque
  std::stack<int, std::vector<int>> s2;  // на основе vector
2. std::queue — FIFO (First In, First Out)
  // Использует push_back и pop_front
  // Обычно на основе deque или list
3. std::priority_queue — очередь с приоритетом
  // На основе vector (max-heap)
  std::priority_queue<int> pq;
  pq.push(3); pq.push(1); pq.push(4);
  pq.top();  // 4 (наибольший элемент)
  pq.pop();  // удаляет 4
```

Особенности адаптеров:

- Не предоставляют итераторы
- Ограниченный интерфейс
- Могут использовать разные базовые контейнеры

Эксперты: Адаптеры демонстрируют принцип композиции в STL

Итераторы: виды и особенности

Итераторы — обобщенные указатели для доступа к элементам контейнеров

Категории итераторов (по возможностям):

1. InputIterator — только чтение, однонаправленный
2. OutputIterator — только запись, однонаправленный
3. ForwardIterator — чтение/запись, однонаправленный
4. BidirectionalIterator — двунаправленный (list, map, set)
5. RandomAccessIterator — произвольный доступ (vector, deque, array)
6. ContiguousIterator (C++20) — непрерывная память (vector, array)

Итераторы: виды и особенности

Примеры итераторов:

```
std::vector<int> vec = {1, 2, 3, 4, 5};

// Получение итераторов
auto begin = vec.begin();    // итератор на первый элемент
auto end = vec.end();        // итератор за последний элемент
auto rbegin = vec.rbegin();  // reverse итератор
auto cbegin = vec.cbegin();  // const итератор (C++11)

*begin = 10;          // разыменование
++begin;              // инкремент
begin == end;         // сравнение
begin + 3;            // только для RandomAccessIterator
```

Инвалидация итераторов:

- vector: при вставке/удалении все итераторы инвалидируются
- deque: инвалидируются при вставке в середине
- list/map: итераторы остаются валидными (кроме удаленного)

Эксперты: Итераторы — это механизм абстракции, который связывает алгоритмы с контейнерами

Современные итераторы (C++20):

- std::ranges::iterator_t
- Концепты итераторов
- Sentinel как альтернатива end()

Часть 2

Итераторы и ассоциативные контейнеры

Адаптеры над итераторами

Адаптеры итераторов — преобразователи, изменяющие поведение итераторов

1. **Reverse итераторы:**

```
std::vector<int> vec = {1, 2, 3, 4, 5};
for (auto it = vec.rbegin(); it != vec.rend(); ++it) {
    std::cout << *it << " ";  // 5 4 3 2 1
}
```

2. **Insert итераторы:**

```
std::vector<int> source = {1, 2, 3};
std::vector<int> dest;

// back_insert_iterator
std::copy(source.begin(), source.end(),
        std::back_inserter(dest));

// front_insert_iterator (для deque, list)
std::deque<int> dq;
std::copy(source.begin(), source.end(),
        std::front_inserter(dq));

// insert_iterator (в указанную позицию)
auto pos = dest.begin() + 1;
std::copy(source.begin(), source.end(),
        std::inserter(dest, pos));
```

3. Move итераторы (C++11):

```
std::vector<std::unique_ptr<Widget>> source, dest;
std::move(source.begin(), source.end(),
        std::back_inserter(dest));
```

4. Stream итераторы:

```
// Чтение из потока
std::istream_iterator<int> input(std::cin), eof;
std::vector<int> numbers(input, eof);

// Запись в поток
std::ostream_iterator<int> output(std::cout, " ");
std::copy(numbers.begin(), numbers.end(), output);
```

Эксперты: Адаптеры итераторов расширяют возможности стандартных алгоритмов

std::list: принцип работы

std::list — двусвязный список

Структура узла:

```
struct Node {
   T data;
   Node* next;
   Node* prev;
};
```

Особенности:

- Быстрая вставка/удаление в любой позиции O(1)
- Медленный произвольный доступ O(n)
- Итераторы остаются валидными после вставки/удаления
- Большой overhead памяти (два указателя на элемент)
- Нет непрерывности памяти → плохая кэш-локальность

Методы list:

```
std::list<int> lst = {1, 2, 3};
// Специфичные методы
lst.push_front(0);          // {0, 1, 2, 3}
lst.pop_front();            // {1, 2, 3}
lst.splice(pos, other_list); // перемещение узлов
lst.merge(other_list);      // слияние отсортированных
lst.unique();               // удаление дубликатов
lst.sort();                 // сортировка
lst.reverse();              // разворот
```

Когда использовать list

1. Частые вставки/удаления в середине контейнера
2. Нужна стабильность итераторов
3. Не нужен произвольный доступ
4. Нужны специфичные операции (splice, merge)

Эксперты: list — это контейнер для случаев, когда важнее стабильность итераторов, чем производительность доступа

Сравнение с vector:

- **Вставка 1000 элементов в середину**: list быстрее в 100-1000 раз
- **Доступ к элементу по индексу**: vector быстрее в 10-100 раз
- **Потребление памяти**: vector эффективнее

std::map: принцип работы

std::map — ассоциативный контейнер, красно-черное дерево

Структура:

```
template<typename Key, typename Value>
class map {
   // Красно-черное дерево
   // Каждый узел: {key, value, color, left, right, parent}
};
```

Особенности:

- Отсортирован по ключу
- Уникальные ключи
- Поиск/вставка/удаление: O(log n)
- Итераторы обходят в порядке возрастания ключей

Примеры:

```
std::map<std::string, int> ages;

// Вставка
ages["Alice"] = 30;         // O(log n)
ages.insert({"Bob", 25});    // O(log n)
ages.emplace("Charlie", 35); // O(log n)
```

```
// Поиск
auto it = ages.find("Alice"); // O(log n), итератор
if (it != ages.end()) {
   std::cout << it->second;  // 30
}

// Обход в отсортированном порядке
for (const auto& [name, age] : ages) {
   std::cout << name << ": " << age << "\n";
}
```

Методы:

- lower_bound(k), upper_bound(k) — границы диапазона
- equal_range(k) — пара итераторов
- count(k) — 0 или 1 (для map)

Эксперты: Используйте map, когда нужен отсортированный ассоциативный контейнер с гарантированной логарифмической сложностью

std::multimap — разрешает дубликаты ключей
std::set — только ключи, без значений

std::unordered_map: принцип работы

std::unordered_map — хэш-таблица

Структура:
- Массив бакетов (buckets)
- Каждый бакет — связный список (цепочка коллизий)
- Коэффициент загрузки (load factor) = size / bucket_count

Особенности:
- Неупорядоченные элементы
- Среднее O(1) для поиска/вставки/удаления
- В худшем случае O(n)
- Требуется std::hash<Key> и operator==

Пример:

```
std::unordered_map<std::string, int> word_count;

// Вставка
word_count["hello"] = 1;          // O(1) среднее
word_count.insert({"world", 2});   // O(1) среднее
word_count.emplace("test", 3);     // O(1) среднее
```

```
// Поиск
auto it = word_count.find("hello"); // O(1) среднее
```

Управление хэш-таблицей:

```
// Установка коэффициента загрузки
word_count.max_load_factor(0.7);

// Резервирование бакетов
word_count.reserve(1000);  // Подготовка для ~1000 элементов

// Рехэширование
word_count.rehash(2000);   // Установить количество бакетов
```

Методы для работы с бакетами:
- bucket_count() — количество бакетов
- bucket_size(n) — размер n-го бакета
- bucket(key) — номер бакета для ключа

Эксперты: Используйте unordered_map, когда порядок не важен,
а важна скорость доступа

Производительность:
- Быстрее map
- В худшем случае медленнее
- Больше накладных расходов памяти
- Чувствителен к качеству хэш-функции

Ключевое слово auto

auto — автоматический вывод типа компилятором (C++11)

Примеры использования:

```
// Простой вывод
auto x = 5;                  // int
auto y = 3.14;               // double
auto str = "hello";          // const char*
auto s = std::string("world"); // std::string

// С контейнерами
std::vector<int> vec = {1, 2, 3};
auto it = vec.begin();       // std::vector<int>::iterator
auto& ref = vec[0];          // int&
const auto& cref = vec[0];   // const int&

// В циклах range-based for (C++11)
for (auto& elem : vec) {     // int&
    elem *= 2;
}

for (const auto& elem : vec) { // const int&
    std::cout << elem << " ";
}
```

```
// С лямбдами
auto lambda = [](int x) { return x * x; };

// decltype(auto) (C++14) — точный вывод типа выражения
decltype(auto) exact = vec[0];  // int&
```

Эксперты: Используйте auto везде, где это упрощает код и предотвращает ошибки типов

Преимущества auto:

1. Упрощает написание кода
2. Предотвращает неявные преобразования
3. Гарантирует инициализацию
4. Работает с типами, которые сложно написать

Когда не использовать auto:

- Когда тип важен для понимания кода
- При использовании proxy-типов (например, vector<bool>)
- Когда нужен определенный тип для перегрузки

Правила вывода типа:

- auto x = expr → вывод как для шаблонного параметра
- auto& x = expr → вывод ссылки
- const auto& x = expr → вывод const ссылки

Ключевое слово decltype

decltype — получение типа выражения без его вычисления (C++11)

Примеры:

```
int x = 10;
decltype(x) y = 20;          // int
decltype((x)) z = x;         // int& (из-за двойных скобок)

std::vector<int> vec = {1, 2, 3};
decltype(vec.begin()) it;    // std::vector<int>::iterator
decltype(vec[0]) ref = vec[0]; // int& (operator[] возвращает ссылку)

// C trailing return type (C++11)
template<typename T, typename U>
auto add(T t, U u) -> decltype(t + u) {
   return t + u;
}

// decltype(auto) (C++14)
template<typename Container, typename Index>
auto access(Container& c, Index i) -> decltype(c[i]) {
   return c[i];  // Возвращает ссылку
}

template<typename Container, typename Index>
decltype(auto) better_access(Container& c, Index i) {
   return c[i];  // То же самое, но короче
}
```

Разница decltype(x) и decltype((x)):

- decltype(x) - тип переменной x
- decltype((x)) - тип выражения (x), может быть ссылкой

Применение:

- Метапрограммирование
- Создание type traits
- Perfect forwarding в шаблонах
- Автоматический вывод возвращаемого типа

Эксперты: decltype особенно полезен в шаблонном коде, где типы могут быть сложными или неизвестными

Пример с контейнерами:

```
template<typename Container>
auto get_value_type(const Container& c)
   -> typename Container::value_type {
   // ...
}
// C decltype
template<typename Container>
auto get_value_type(const Container& c)
   -> decltype(*c.begin()) {
   // Автоматический вывод типа элементов
   return *c.begin();
}
```

Примеры использования auto и decltype

Примеры:

```
// 1. Работа с итераторами
std::map<std::string, int> scores;
// Без auto
std::map<std::string, int>::iterator it = scores.find("Alice");
// С auto
auto it = scores.find("Alice");  // Короче и безопаснее

// 2. Range-based for с структурированными привязками (C++17)
std::map<std::string, int> ages = {{"Alice", 30}, {"Bob", 25}};
for (const auto& [name, age] : ages) {  // Автоматический вывод
    std::cout << name << ": " << age << "\n";
}

// 3. Обобщенные лямбды (C++14)
auto print_container = [](const auto& container) {
    for (const auto& elem : container) {
        std::cout << elem << " ";
    }
    std::cout << "\n";
};
```

```
// 4. decltype для метапрограммирования
template<typename T>
auto make_vector(T value) {
    std::vector<decltype(value)> vec;  // Вектор типа T
    vec.push_back(value);
    return vec;
}

// 5. Perfect forwarding с decltype
template<typename T>
auto make_unique_ptr(T&& arg)
    -> std::unique_ptr<decltype(std::forward<T>(arg))> {
    return std::make_unique<decltype(std::forward<T>(arg))>(
        std::forward<T>(arg));
}

// 6. Вывод типа сложного выражения
template<typename T>
auto process(const std::vector<T>& vec)
    -> decltype(vec[0] * vec.size()) {
    if (vec.empty()) return {};
    return vec[0] * vec.size();
}
```

Рекомендации экспертов:

- auto для итераторов и вызовов алгоритмов
- decltype для метапрограммирования
- Предпочитайте auto явному указанию типа, когда это упрощает код

Часть 3

Современный C++

Современные подходы к работе с контейнерами

Рекомендации от экспертов:

Скотт Мейерс (Effective STL):
1. Предпочитайте vector другим контейнерам
2. Используйте reserve() для предотвращения реаллокаций
3. Правильно выбирайте между map и unordered_map
4. Используйте swap для быстрой очистки контейнеров
5. Избегайте vector<bool> (используйте bitset или deque<bool>)

Андрей Александреску (Modern C++ Design):
1. Используйте policy-based design для создания гибких контейнеров
2. Применяйте type lists для работы с гетерогенными контейнерами
3. Создавайте собственные аллокаторы для специфичных задач
4. Используйте compile-time проверки для безопасности типов

Николаус Джосуттис (The C++ Standard Library):
1. Понимайте гарантии сложности операций
2. Знайте, когда итераторы инвалидируются
3. Используйте правильные методы поиска (find vs count)
4. Оптимизируйте на основе реальных измерений

Современные возможности C++17/20:

1. std::string_view для работы со строками без копирования
2. Structured bindings для удобной работы с парами
3. std::optional для опциональных значений
4. std::variant для безопасных объединений
5. Ranges (C++20) для работы с контейнерами

Пример с ranges (C++20):

```
#include <ranges>
std::vector<int> vec = {1, 2, 3, 4, 5};
auto result = vec | std::views::filter([](int x) { return x % 2 == 0; })
                  | std::views::transform([](int x) { return x * 2; });
```

Пример: оптимизированный кэш с использованием STL

```
template<typename Key, typename Value>
class LRUCache {
    using ListType = std::list<std::pair<Key, Value>>;
    using MapType = std::unordered_map<Key, typename ListType::iterator>;
    ListType items;                    // Хранение пар ключ-значение
    MapType lookup;                    // Быстрый поиск по ключу
    size_t capacity;
  public:
    explicit LRUCache(size_t cap) : capacity(cap) {}
    // Вставка с auto и perfect forwarding
    template<typename K, typename V>
    void insert(K&& key, V&& value) {
        auto it = lookup.find(key);
       if (it != lookup.end()) {
            // Перемещаем в начало (последний использованный)
            items.splice(items.begin(), items, it->second);
            it->second->second = std::forward<V>(value);
            return;
        }
        // Проверка capacity
        if (items.size() >= capacity) {
            // Удаляем самый старый (конец списка)
            auto last = std::prev(items.end());
            lookup.erase(last->first);
            items.pop_back();
        }
        // Вставка в начало
        items.emplace_front(std::forward<K>(key), std::forward<V>(value));
        lookup.emplace(std::forward<K>(key), items.begin());
    }
```

```
    // Поиск с optional (C++17)
    std::optional<Value> get(const Key& key) {
        auto it = lookup.find(key);
        if (it == lookup.end()) {
            return std::nullopt;
        }

        // Перемещаем в начало
        items.splice(items.begin(), items, it->second);
        return it->second->second;
    }

    // Использование decltype для интерфейса
auto begin() -> decltype(items.begin()) { return items.begin(); }
auto end() -> decltype(items.end()) { return items.end(); }

    size_t size() const { return items.size(); }
    bool empty() const { return items.empty(); }
};

// Использование
LRUCache<std::string, int> cache(100);
cache.insert("key1", 42);
if (auto value = cache.get("key1")) {
    std::cout << *value << "\n";  // 42
}
```

```cpp
#include <iostream>
#include <list>
#include <unordered_map>
#include <optional>

// LRU Cache — хранит последние использованные элементы
template<typename Key, typename Value>
class LRUCache {
   using ListType = std::list<std::pair<Key, Value>>;
   using MapType = std::unordered_map<Key, typename ListType::iterator>;

   ListType items;     // // Хранение пар ключ-значение. порядок использования: начало — свежие, конец — старые
   MapType lookup;     // быстрый поиск по ключу
   size_t capacity;
 public:
   explicit LRUCache(size_t cap) : capacity(cap) {}

   // Вставка с perfect forwarding
   template<typename K, typename V>
   void insert(K&& key, V&& value) {
      auto it = lookup.find(key);

      if (it != lookup.end()) {
         // Ключ есть — обновляем значение и перемещаем в начало  (последний использованный)

         items.splice(items.begin(), items, it->second);
         it->second->second = std::forward<V>(value);
         return;
      }
// Проверка capacity
                    // Если переполнено — удаляем самый старый
      if (items.size() >= capacity) {
         auto last = std::prev(items.end());
         lookup.erase(last->first);
         items.pop_back();
      }
            // Вставляем новый элемент в начало
      items.emplace_front(std::forward<K>(key), std::forward<V>(value));
      lookup.emplace(std::forward<K>(key), items.begin());  }

// Поиск — перемещает найденный элемент в начало
   std::optional<Value> get(const Key& key) {
      auto it = lookup.find(key);
      if (it == lookup.end())
         return std::nullopt;

      items.splice(items.begin(), items, it->second);
      return it->second->second;
   }
   // Итераторы для обхода
   auto begin() { return items.begin(); }
   auto end()   { return items.end(); }

   size_t size()  const { return items.size(); }
   bool empty()   const { return items.empty(); }
};

int main() {
   LRUCache<std::string, int> cache(3);

   cache.insert("one", 1);
   cache.insert("two", 2);
   cache.insert("three", 3);
   cache.insert("four", 4);  // вытесняет "one"

   if (auto val = cache.get("two")) {
      std::cout << "two: " << *val << std::endl;  // 2
   }

   if (auto val = cache.get("one")) {
      std::cout << "one: " << *val << std::endl;
   } else {
      std::cout << "one not found" << std::endl;  // вытеснен
   }

   std::cout << "\nCache content (most recent first):" << std::endl;
   for (const auto& [key, val] : cache) {
      std::cout << "  " << key << ": " << val << std::endl;
   }
}
```

Your Output

```
two: 2
one not found

Cache content (most recent first):
  two: 2
  four: 4
  three: 3
```

STL - набор контейнеров для разных задач:

- vector — для большинства случаев
- deque — когда нужны операции с обоих концов
- list — когда важна стабильность итераторов
- map/set — для отсортированных данных
- unordered_map/set — для максимальной скорости доступа

2. **Итераторы связывают контейнеры и алгоритмы:**

- Разные категории для разных нужд
- Адаптеры расширяют возможности
- Понимание инвалидации критически важно

3. **Современные возможности C++ упрощают работу:**

- auto и decltype для вывода типов
- Range-based for для итерации
- Structured bindings для работы с парами
- Ranges (C++20) для выразительных операций

4. **Рекомендации экспертов:**

- Мейерс: предпочитайте vector, используйте reserve()
- Александреску: создавайте гибкие компоненты
- Джосуттис: понимайте сложность операций

«Искусство работы с STL —
в правильном выборе контейнера
и понимании его внутреннего устройства»
— Николаус Джосуттис